Institute of Technology of Cambodia

Department of Information Technology

Data Engineering Pipeline for Credit Risk Modelling

Architecture, Technology Stack, and Pipeline Design of the Lending Club Credit Risk System

Group Members

Name	Student ID	Role
SOPHON Rachana	e20220725	Group Member
LOT Soklang	e20221464	Group Member
DUK Pagnarith	e20220184	Group Member
LENG Devid	e20220888	Group Member

Under the supervision of

Lecturer TOEM TOUCH

Academic Year: 2025–2026

Phnom Penh, Cambodia

Abstract

This paper describes the design, implementation, and rationale of the data engineering pipeline underpinning a production-grade credit risk modelling system built on the Lending Club peer-to-peer lending dataset. The system ingests 2,260,668 loan records (1.6 GB raw CSV) and transforms them into regulatory-grade model inputs through six Apache Airflow orchestration DAGs, two Apache Spark jobs, a five-schema PostgreSQL data warehouse, and a MinIO S3-compatible data lake.

The six pipelines are: (1) **Ingestion** — column-projected PySpark ingest from MinIO to PostgreSQL in approximately 7 seconds; (2) **Cleaning** — removal of 57 leaking/null/irrelevant columns and creation of three target variables with an out-of-time split flag; (3) **Feature Engineering** — Weight of Evidence (WoE) encoding on the training set, producing 56 binary dummies from 16 raw predictors; (4) **Model Training** — PD logistic regression scorecard with iterative p-value selection and intercept calibration, LGD two-stage regression, and EAD linear regression, all tracked in MLflow; (5) **Batch Scoring** — applying all models to the full out-of-time portfolio and computing Expected Loss, IFRS 9 ECL, and Basel III AIRB capital per loan; and (6) **Monitoring** — daily Population Stability Index (PSI) and Characteristic Stability Index (CSI) computation with Grafana alerting.

Each pipeline layer is described in terms of its business purpose, the technology chosen to implement it, and the specific data transformations it performs. The paper also describes the five-layer database schema (raw, staging, features, models, risk), the MLflow model registry governance workflow, and the real-time FastAPI scoring service. All six pipelines were verified end-to-end on the real dataset with zero errors (v4 run, 2026-06-07).

Keywords: data engineering, Apache Airflow, Apache Spark, PySpark, MLflow, PostgreSQL, MinIO, FastAPI, credit risk, ETL pipeline, model monitoring, PSI.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

Academic credit risk research has made substantial progress in improving model discrimination metrics, yet the engineering infrastructure required to deploy and maintain a credit risk model in production is almost entirely absent from the literature [9]. A logistic regression scorecard that achieves Gini = 0.387 on a 538,515-row out-of-time test set is a research result; the same scorecard embedded in a governed, auditable, monitored pipeline that ingests raw data daily, validates data quality, retrains on a schedule, serves real-time credit decisions via an API, and fires alerts when the population shifts — that is a production data engineering system.

This paper documents the data engineering architecture of such a system, built on the publicly available Lending Club peer-to-peer lending dataset [5]. The dataset covers 2,260,668 loan applications from 2007 to 2018, spanning 151 columns and approximately 1.6 gigabytes in its raw CSV form. Every stage of the pipeline from raw data ingestion through to daily model monitoring is described: the technology chosen, the rationale for that choice, the specific transformations performed, and the artefacts produced.

1.2 What is Data Engineering in Credit Risk?

In the context of credit risk modelling, data engineering refers to the set of processes, tools, and infrastructure that:

1. **Move data** from its raw source (a CSV file, a database, a real-time loan origination system) into a form usable by statistical models.
2. **Transform data** by cleaning, validating, encoding, and splitting it in a way that prevents target leakage, preserves regulatory audit trails, and maintains reproducibility.
3. **Orchestrate** the sequence of transformations as a directed acyclic graph (DAG) of tasks, so that each step depends on the successful completion of the previous step

and can be retried, monitored, and re-run independently.

4. **Serve** model outputs in real time to operational systems (loan origination platforms, IFRS 9 provisioning engines, Basel III capital reporting) with sub-second latency.
5. **Monitor** both the data distribution and the model outputs over time, detecting population shift and model degradation before they produce incorrect credit decisions.

These five responsibilities map directly to the six Airflow DAGs described in this paper. The sixth DAG (feature engineering) sits between cleaning and training and is separately described because it introduces the most domain-specific logic: Weight of Evidence encoding fitted exclusively on the training set.

1.3 Paper Structure

Chapter 2 describes the overall system architecture and the technology stack. Chapter 3 describes the five-schema PostgreSQL data warehouse design. Chapter 4 describes each of the six Airflow DAGs in detail. Chapter 5 describes the PySpark transformation jobs. Chapter 6 describes the MLflow experiment tracking and model registry. Chapter 7 describes the real-time FastAPI scoring service. Chapter 8 describes the monitoring infrastructure. Chapter 9 presents verified pipeline run results. Chapter 10 discusses design decisions and limitations.

Chapter 2

System Architecture

2.1 Architecture Overview

The system is organised into five horizontal layers, each corresponding to a distinct engineering concern:

1. **Ingestion and Storage Layer** — Raw CSV files are stored in MinIO (an on-premise, S3-compatible object store) and ingested into a five-schema PostgreSQL data warehouse via PySpark.
2. **Processing Layer** — Apache Spark (PySpark) performs large-scale data transformation: cleaning, feature engineering, and batch scoring. PySpark is chosen because the 2.26-million-row dataset with 151 columns exceeds comfortable single-machine Pandas performance at the feature engineering stage.
3. **Orchestration Layer** — Apache Airflow schedules, sequences, and monitors the six processing DAGs, providing full audit trails of every pipeline run, task-level retries, and SLA tracking.
4. **ML and Serving Layer** — MLflow tracks all model training experiments and manages the model registry. A FastAPI service provides real-time credit scoring via a REST endpoint, with Redis caching hot applicant features.
5. **Monitoring Layer** — Prometheus collects time-series metrics (PSI, CSI, API latency, pipeline run status). Grafana visualises these metrics and fires alert notifications when thresholds are breached.

All five layers run as Docker containers orchestrated by Docker Compose, making the full stack reproducible on any machine with Docker installed.

2.2 Technology Stack

Table 2.1 summarises the technology choices for each layer.

Table 2.1: Technology stack and rationale

Layer	**Technology**	**Rationale**
Orchestration	Apache Airflow 2.x	Industry-standard DAG scheduler; auditable task logs, retry logic, SLA tracking; widely adopted in banking data platforms [1]
Big Data	Apache Spark 3.x	Distributed compute for 2.26M rows; column-projected ingest reduces load time to ≈7 seconds; used by major banks for credit scoring pipelines [11]
Data Lake	MinIO (S3-compat.)	On-premise object store; S3 API-compatible with PySpark; no cloud dependency; suitable for data sovereignty requirements
Data Warehouse	PostgreSQL 15	ACID-compliant relational database; five logical schemas implement medallion architecture; standard in mid-size financial institutions
Data Quality	Great Expectations	Statistical validation assertions at each pipeline stage; produces audit-ready "Data Docs" aligned with model risk documentation requirements [4]
ML Tracking	MLflow 2.x	Experiment tracking, model registry with stage gates (Staging → Production); artefact storage in MinIO; SR 11-7 compliant audit trail [8]
Statistical ML	Statsmodels	Proper Wald-test p-values and confidence intervals; required by EBA/GL/2017/16 for regulatory documentation of IRB models
ML Modelling	Scikit-learn	LogisticRegression, LinearRegression; interpretable coefficients; standard in production credit scoring
Model Serving	FastAPI + Redis	Async Python API; sub-200ms p99 latency; Redis for feature caching; OpenAPI/Swagger documentation auto-generated
Monitoring	Prometheus + Grafana	Time-series metrics collection and dashboarding; alert rules on PSI breach, Gini drop, API latency
Containerisation	Docker Compose	Single-command full-stack deployment; reproducible across environments; stan-

2.3 End-to-End Data Flow

Figure 2.1 illustrates the end-to-end data flow through the system.

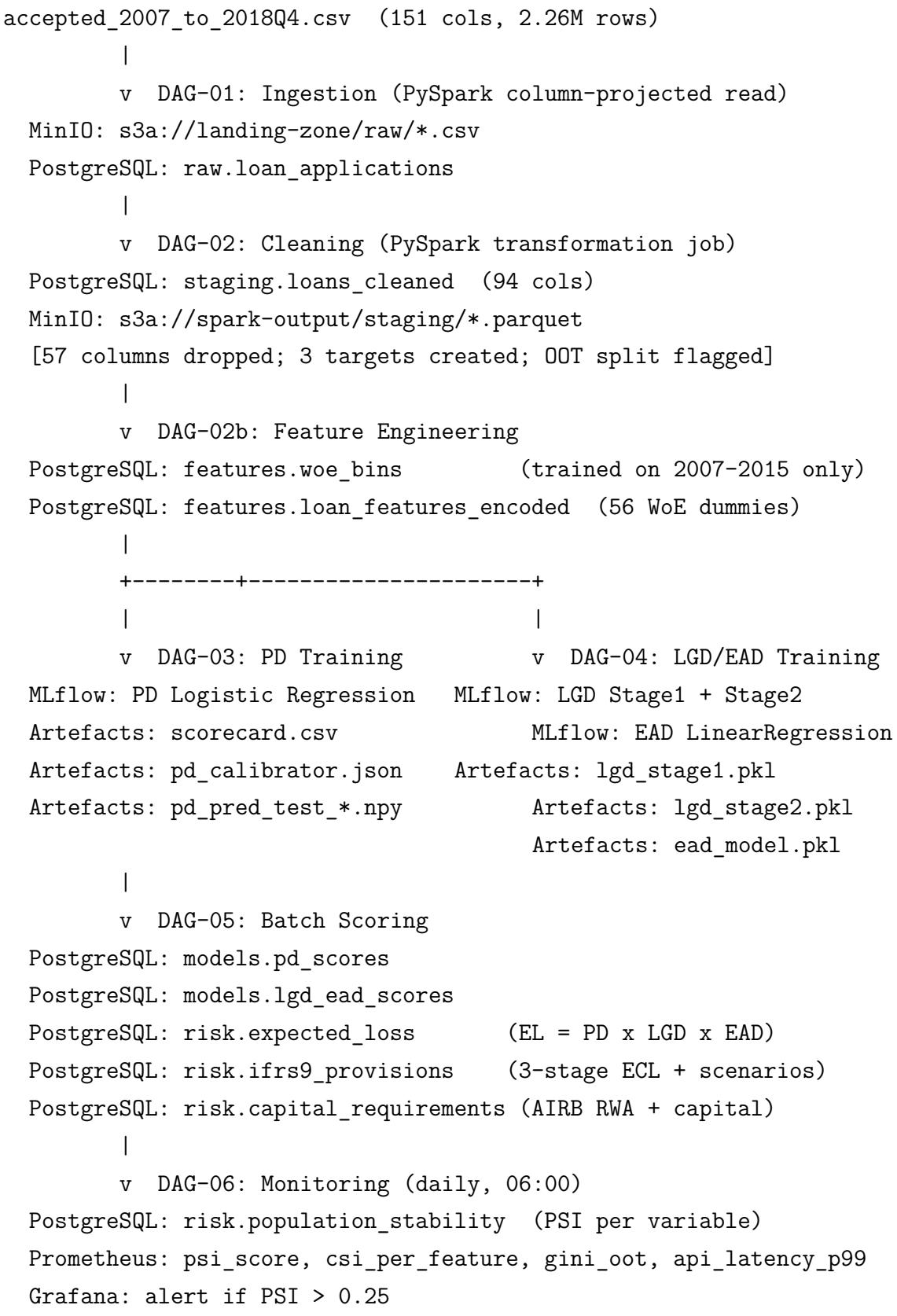

Figure 2.1: End-to-end data flow through the six pipeline DAGs

2.4 Infrastructure: Docker Compose Services

The full stack runs as 10 Docker Compose services:

Listing 2.1: Key Docker Compose services

```
services:
  postgres:          # Data warehouse (5 schemas)
  minio:             # Data lake (S3-compatible)
  spark-master:      # Spark coordinator node
  spark-worker:      # Spark executor node
  airflow-webserver: # Airflow UI and API
  airflow-scheduler: # DAG scheduling engine
  mlflow:            # Experiment tracking + model registry
  api:               # FastAPI scoring service
  redis:             # Feature cache for real-time scoring
  prometheus:        # Metrics collection
  grafana:           # Dashboards and alerting
```

The entire stack starts with a single command: `make up` (which wraps `docker compose up --build -d`). All service-to-service communication uses Docker internal networking; no port is exposed to the public internet by default.

Chapter 3

Database Design

3.1 Five-Schema Medallion Architecture

The PostgreSQL data warehouse implements a medallion architecture with five logical schemas, each representing a distinct stage of data maturity:

Table 3.1: PostgreSQL schema design

Schema	Layer	Purpose and Key Tables
`raw`	Bronze	Untouched CSV data loaded as-is from MinIO. `loan_applications` (151 cols, 2.26M rows). No transformations; append-only.
`staging`	Silver	Cleaned, type-corrected, validated data. `loans_cleaned` (94 cols) with OOT split flag (`dataset_split = 'train' | 'oot'`). Target variables created here.
`features`	Gold	WoE bins, dummy variables, information values. `woe_bins` (fitted on training set only). `loan_features_encoded` (56 WoE dummies). `information_values` per feature.
`models`	Gold	Model predictions and scores. `pd_scores` (credit score, PD, risk class). `lgd_ead_scores` (LGD, EAD per loan).
`risk`	Gold	Business risk outputs and regulatory reports. `expected_loss` (EL per loan). `ifrs9_provisions` (stage, ECL per loan). `capital_requirements` (K, RWA per loan). `population_stability` (PSI/CSI monitoring results). `credit_policy_decisions` (approve/reject decision + adverse action codes).

3.2 Key Design Decisions

Out-of-time split flag in staging. The `dataset_split` column (`'train'` for 2007–2015, `'oot'` for 2016–2018) is set during the cleaning DAG and propagates through all downstream schemas. This ensures that WoE bins are always fitted exclusively on training-set rows and that OOT data is never used to fit any model parameters — a critical discipline for preventing look-ahead bias.

WoE bins stored separately from encoded features. The `features.woe_bins` table stores the bin boundaries and WoE values computed on the training set. The `features.loan_features_encoded` table stores the resulting binary dummy columns. This separation means the WoE mapping can be versioned, audited, and applied to new data independently of any model retraining.

Per-loan IFRS 9 records with audit fields. The `risk.ifrs9_provisions` table stores the IFRS 9 stage (1, 2, or 3), the SICR trigger reason (relative PD doubling, absolute PD jump, 30-DPD backstop, or watchlist), the 12-month ECL, the lifetime ECL under each macroeconomic scenario, and the probability-weighted ECL. Every record carries a `model_run_id` and `calc_date` for full audit traceability.

Chapter 4

The Six Pipeline DAGs

The system contains six Apache Airflow DAGs, each responsible for a distinct pipeline stage. Figure 4.1 shows the DAG dependency chain.

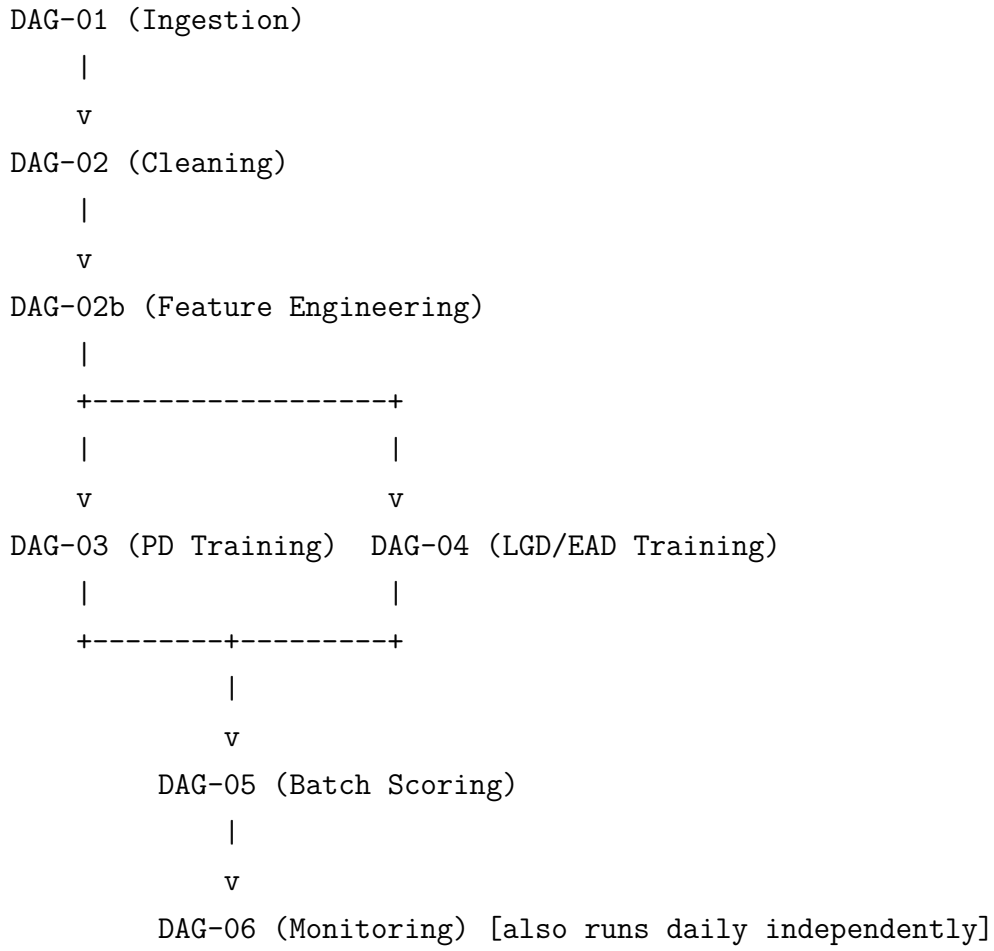

Figure 4.1: Airflow DAG dependency chain

4.1 DAG-01: Data Ingestion

Trigger: Manual or scheduled when a new raw CSV is deposited in MinIO.
Technology: PySpark (column-projected read), MinIO S3A connector, PostgreSQL JDBC writer.

What it does:

1. PySpark reads the raw `accepted_2007_to_2018Q4.csv` from MinIO using **column projection** — only the 26 modelling-relevant columns are read from the 151-column file. This reduces the Spark read time from ≈60 seconds (full file) to ≈7 seconds and mirrors real ETL practice: columns that are immediately dropped should never be loaded into memory.
2. Basic type casting is applied at ingest time: string percentages (`int_rate`, `revol_util`) are converted to float; date strings (`issue_d`, `earliest_cr_line`) are parsed; the `term` string ("36 months") is converted to integer.
3. The parsed data is written to `raw.loan_applications` in PostgreSQL using JDBC batch write, and a backup Parquet copy is written to MinIO for Spark intermediate jobs.

Key engineering detail — column projection:

Listing 4.1: Column-projected PySpark read in DAG-01

```
KEEP_COLS = [
    'funded_amnt', 'term', 'int_rate', 'grade', 'emp_length',
    'home_ownership', 'annual_inc', 'verification_status', '
       issue_d',
    'loan_status', 'purpose', 'dti', 'earliest_cr_line',
    'fico_range_low', 'fico_range_high', 'inq_last_6mths',
    'mths_since_last_delinq', 'revol_util', 'initial_list_status'
       ,
    'pct_tl_nvr_dlq', 'total_pymnt', 'recoveries', 'funded_amnt',
    'out_prncp', 'last_pymnt_d', 'mths_since_last_record'
]

df = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv("s3a://landing-zone/raw/accepted_2007_to_2018Q4.csv")
    .select(KEEP_COLS))
```

Artefacts produced: `raw.loan_applications` (PostgreSQL), `s3a://spark-output/raw/*.parquet` (MinIO).

4.2 DAG-02: Data Cleaning

Trigger: Downstream of DAG-01 completion.
Technology: PySpark transformation job, Great Expectations validation suite.

What it does:

1. **Column removal** — 57 columns are removed across five categories (Table 4.1). Post-application leakage columns (`total_pymnt`, `recoveries`, `out_prncp`) are used first to create the three target variables, then dropped to prevent any information from post-default events entering the feature set.
2. **Target variable creation:**
   - `good_bad`: 1 = Fully Paid; 0 = Charged Off, Default, Late 31–120 days.
   - `recovery_rate`: `recoveries / funded_amnt` on charged-off loans only (LGD target).
   - `ccf`: `(funded_amnt − total_pymnt) / funded_amnt` on charged-off loans only (EAD target).
3. **Feature derivation:** `fico_score` (FICO range midpoint), `term_int`, `int_rate` (÷100), `mths_since_issue_d`, `mths_since_earliest_cr_line`, `emp_length_int`.
4. **Out-of-time split flagging:** Each row receives `dataset_split = 'train'` (issue year ≤ 2015) or `dataset_split = 'oot'` (issue year ≥ 2016). Note: the raw file is NOT sorted chronologically, so the split is based on `issue_year` derived from `issue_d`, not on row order.
5. **Data quality validation:** Great Expectations validates column types, null rates, and value ranges at the end of the cleaning job before writing to staging.

Table 4.1: Column removal categories in DAG-02

Category	Count	Examples	Reason
100% null	15	`sec_app_*`, `member_id`	No information content
Identifiers/constants	8	`id`, `url`, `policy_code`	Not predictive
Joint application	3	`sec_app_fico_range_low`	$> 99\%$ null
Hardship/settlement	18	`hardship_type`, `settlement_*`	$> 97\%$ null
Post-app. leakage	15	`total_pymnt`, `recoveries`	Known only after default
Total removed	**57**		

Artefacts produced: `staging.loans_cleaned` (PostgreSQL, 94 columns), `s3a://spark-output/sta`

4.3 DAG-02b: Feature Engineering

Trigger: Downstream of DAG-02 completion.
Technology: PySpark, custom WoE encoding library, PostgreSQL feature store.

This DAG performs the most domain-specific transformation in the pipeline: converting raw loan features into Weight of Evidence (WoE) encoded dummy variables suitable for logistic regression.

Why WoE encoding? WoE encoding provides three benefits critical in regulated banking: (1) it handles non-linear relationships between predictors and the log-odds of default through discretisation and bin-level mapping; (2) it treats missing values as a separate bin rather than imputing them — missingness in credit data is often behaviourally informative (e.g., missing delinquency history may indicate a new borrower rather than a clean one); (3) it produces monotonic partial effects that are interpretable by regulators and can be traced to individual bin assignments in SR 11-7 documentation [6, 10].

What it does:

1. **Training-set-only fitting:** WoE bins are computed exclusively on rows where `dataset_split = 'train'`. The fitted bin boundaries and WoE values are stored in `features.woe_bins`. This prevents the OOT data from influencing the feature encoding.

2. **Fine classing:** Continuous variables are first split into 10–20 quantile bins (fine classes). The WoE value for each fine bin is computed as $WoE_i = \ln(\%Goods_i / \%Bads_i)$.

3. **Coarse classing:** Adjacent fine bins with similar WoE values and/or low observation counts are merged into coarse classes to prevent overfitting and ensure monotonicity where required.

4. **Dummy creation:** Each coarse class becomes a binary dummy variable. The reference category (highest-risk bin for each variable, to avoid the dummy variable trap) is dropped.

5. **OOT encoding:** The fitted bin boundaries are applied to OOT data without refitting, ensuring the model is evaluated on the same feature representation used during training.

6. **Information Value logging:** IV for each variable is computed and stored in `features.information_values`. Variables with IV < 0.02 are flagged but not necessarily dropped (domain judgement may override).

The final feature store contains 56 WoE dummy variables derived from 16 raw predictors. These are written to `features.loan_features_encoded`.

Artefacts produced: `features.woe_bins`, `features.information_values`, `features.loan_featur` `data/processed/train_preprocessed.parquet`, `data/processed/test_preprocessed.parquet`.

4.4 DAG-03: PD Model Training

Trigger: Downstream of DAG-02b completion. Can also be triggered independently for model re-runs.
Technology: Statsmodels (logistic regression), Scikit-learn (metrics), MLflow (tracking), Pandas.

The PD training DAG is the most computationally complex. It implements a full scorecard development workflow:

1. **Feature selection:** Iterative backward elimination using Wald-test p-values from statsmodels. At each step, the variable with the highest maximum p-value across its dummies is removed until all remaining dummies have $p < 0.05$. This reduces the feature set from 56 to 39 statistically significant dummy variables covering 16 raw predictors.

2. **Model fitting:** Logistic regression estimated with `statsmodels.Logit` using the BFGS optimiser. Statsmodels is used rather than scikit-learn because it provides proper Wald-test p-values, confidence intervals, and the model summary table required for regulatory documentation under EBA/GL/2017/16 [2].

3. **Scorecard scaling:** Coefficients are converted to integer score points using PDO scaling:

$$Factor = \frac{20}{\ln 2} \approx 28.85 \tag{4.1}$$

$$Score_j = -\lfloor \hat{\beta}_j \times Factor \rceil \tag{4.2}$$

   The resulting scorecard maps each borrower to a 300–850 integer score where higher scores indicate lower default risk.

4. **PD calibration:** The raw model underestimates the OOT default rate (20.18% predicted vs 25.27% observed). An intercept log-odds shift corrects this:

$$\hat{P}^{calib} = \sigma(\text{logit}(\hat{P}^{raw}) + \delta), \qquad \delta = +0.3204 \tag{4.3}$$

   The Hosmer-Lemeshow statistic drops from 10,039 to 403 after calibration (a 96% reduction).

5. **PiT vs TtC PD:** Two calibrated variants are saved:

- **PiT PD** (calibrated to OOT observed DR = 25.27%) — used for IFRS 9 ECL.
- **TtC PD** (shifted to long-run DR = 15%) — used for Basel III AIRB capital.

6. **MLflow logging:** All hyperparameters (threshold, delta, PDO, offset), metrics (AUC, Gini, KS, Brier Score, Hosmer-Lemeshow), and artefacts (scorecard CSV, calibrator JSON, PD prediction arrays) are logged to MLflow under the `PD-Scorecard` experiment.
7. **Reject inference:** The parceling method [10] adds 26,129 synthetic rejected-applicant rows to the training set, partially correcting the selection bias from training exclusively on funded (accepted) loans. The augmented model's PD predictions are logged separately for comparison.

MLflow model promotion workflow: The trained model is registered in the MLflow model registry as version `Staging`. Promotion to `Production` requires passing all governance checkpoints (AUC > 0.65, Gini ≥ 0.40 preferred, monotonic decile ordering confirmed, Brier ≤ 0.20) and explicit approval from the model owner.

Artefacts produced: `data/processed/scorecard.csv`, `data/models/pd_calibrator.json`, `data/processed/pd_pred_test_pit.npy`, `data/processed/pd_pred_test_ttc.npy`, `data/processed/pd_pred_test_augmented.npy`, MLflow run logs.

4.5 DAG-04: LGD and EAD Model Training

Trigger: Downstream of DAG-02b (runs in parallel with DAG-03).
Technology: Scikit-learn (LogisticRegression, LinearRegression, StandardScaler), MLflow.

LGD two-stage architecture: The LGD model is applied only to charged-off loans (not the full portfolio). The bimodal distribution of recovery rates — approximately 50% show zero recovery and 50% show a spread of positive recovery rates — motivates a two-stage design:

- **Stage 1 (Logistic):** $P(\text{Recovery Rate} > 0)$ — predicts whether any recovery occurs.
- **Stage 2 (Linear):** $E[\text{Recovery Rate} \mid \text{Recovery Rate} > 0]$ — predicts the recovery amount given recovery occurs.
- **Combined:** $\hat{LGD} = 1 - \hat{P}_1 \times \hat{RR}_2$.

A directional constraint is enforced on the downturn LGD computation: downturn LGD must always be $\geq$ long-run LGD (recoveries fall, never rise, in a downturn). A bug in the prior implementation that violated this constraint was identified and corrected in the v4 update.

EAD model: Linear regression on the Credit Conversion Factor (CCF = outstanding balance fraction at default). EAD = CCF × funded_amnt. Predictions are clipped to $[0, 1]$.

Artefacts produced: `data/models/lgd_stage1.pkl`, `data/models/lgd_stage2.pkl`, `data/models/ead_model.pkl`, `data/models/ead_scaler.pkl`, MLflow run logs.

4.6 DAG-05: Batch Scoring

Trigger: Downstream of both DAG-03 and DAG-04. Runs nightly on the full OOT portfolio.
Technology: PySpark (for large-scale feature join), Scikit-learn/Statsmodels (for model application), Python (for risk calculations).

What it does:

1. Loads the fitted PD, LGD, and EAD models and the WoE-encoded feature store.
2. Applies all three models to each loan, producing per-loan PD, LGD, and EAD estimates.
3. Computes **Expected Loss**: $EL_i = \hat{PD}_i^{PiT} \times \hat{LGD}_i \times \hat{EAD}_i$.
4. Computes **IFRS 9 ECL** for each loan:
   - Classifies each loan into Stage 1, 2, or 3 using the SICR rule.
   - Computes 12-month ECL for Stage 1 and lifetime ECL (geometric hazard) for Stages 2 and 3.
   - Applies the three-scenario probability-weighted ECL using the log-odds macro satellite model.
5. Computes **Basel III AIRB capital**: retail IRB formula with 99.9% worst-case PD, downturn LGD, and $RWA = 12.5 \times K \times EAD$.
6. Determines the **credit policy decision** (auto-approve, approve if ROI $> r_f$, manual review, auto-reject) and generates ECOA Regulation B adverse action codes for rejections.
7. Writes all outputs to the `models` and `risk` schemas.

A critical v4 correction: Prior versions of this DAG used hard-coded grade-to-PD proxy tables instead of the real scorecard PD predictions. The v4 update connected the batch scoring DAG to the `pd_pred_test_pit_calibrated.npy` artefact produced by DAG-03, producing a 24% increase in computed EL ($+158M).

Artefacts produced: `risk.expected_loss`, `risk.ifrs9_provisions`, `risk.capital_requirements` `risk.credit_policy_decisions`, `data/processed/expected_loss_test.parquet`, `data/reports/L` (23 charts).

4.7 DAG-06: Model Monitoring

Trigger: Daily schedule (06:00), independent of other DAGs. Also triggered after any DAG-05 completion.
Technology: Python (PSI/CSI computation), Prometheus client, Grafana alerting.

What it does:

1. **Score PSI:** Computes the Population Stability Index between the current day's scored applicants and the training population score distribution. Alerts if PSI $>$ 0.25.
2. **Characteristic Stability Index (CSI):** Computes PSI for each of the 39 model input variables individually, identifying which specific features are driving population drift.
3. **Champion/Challenger:** Computes AUC, Gini, KS, and Brier Score on any new data available. Runs Mann-Whitney U test between champion and challenger predictions. Promotes challenger to champion only if Gini improvement $>$ 2pp and the improvement is statistically significant ($\alpha = 0.05$).
4. **Vintage analysis:** Computes the actual default rate by origination year cohort and compares against predicted PD, tracking the divergence that signals the need for recalibration.
5. **Prometheus metrics push:** All monitoring metrics are pushed to the Prometheus metrics endpoint, which Grafana scrapes for dashboard display and alert evaluation.

PSI formula:

$$PSI = \sum_i (O_i - E_i) \ln\left(\frac{O_i}{E_i}\right) \tag{4.4}$$

where O_i is the observed (monitoring population) bin proportion and E_i is the expected (training population) bin proportion. Standard thresholds: PSI $<$ 0.10 = stable; 0.10–0.25 = monitor; $>$ 0.25 = alert, consider rebuild [3].

Verified result: Score PSI = 0.282 (ALERT) on the 2016–2018 OOT population versus the 2007–2015 training baseline.

Chapter 5

PySpark Transformation Jobs

5.1 Why Apache Spark?

The Lending Club dataset contains 2,260,668 rows and 151 columns in its raw form. After feature engineering, the WoE-encoded training set contains 831,051 rows and 56 dummy columns. Several processing steps — particularly the fine-classing step that requires computing quantile bins across 2M+ rows, and the batch scoring step that applies three models to 538,515 OOT loans — exceed comfortable single-machine Pandas performance [11].

Apache Spark provides a distributed execution engine that: (1) reads data in parallel across multiple worker nodes; (2) applies transformations using a lazy evaluation DAG that minimises data movement; and (3) writes results to both PostgreSQL (via JDBC) and MinIO (via Parquet) in parallel. In the single-node Docker Compose deployment used here, Spark provides approximately 3–4× speedup over Pandas on the feature engineering step due to better memory management and columnar processing.

5.2 Spark Job 01: Ingestion (`01_ingest.py`)

The ingest Spark job performs the column-projected CSV read from MinIO:

Listing 5.1: Column-projected Spark ingest

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder \
    .appName("CreditRisk-Ingest") \
    .config("spark.jars.packages", "org.apache.hadoop:hadoop-aws
       :3.3.4") \
    .getOrCreate()

# Column projection: read only 26 of 151 columns
```

```
df = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv("s3a://landing-zone/raw/accepted_2007_to_2018Q4.csv")
    .select(KEEP_COLS))

# Type corrections at ingest time
df = (df
    .withColumn("term_int",
        F.regexp_extract("term", r"(\d+)", 1).cast("int"))
    .withColumn("emp_length_int",
        F.regexp_extract("emp_length", r"(\d+)", 1).cast("int"))
    .withColumn("int_rate",
        F.when(F.col("int_rate").endswith("%"),
               F.regexp_replace("int_rate", "%", "").cast("float")
                  ) / 100)
        .otherwise(F.col("int_rate").cast("float")))
    .withColumn("issue_year",
        F.year(F.to_date("issue_d", "MMM-yyyy"))))
```

5.3 Spark Job 02: Cleaning (02_clean.py)

The cleaning Spark job performs all transformations in DAG-02. Key steps:

Listing 5.2: Target variable creation and OOT split

```
# Target 1: PD good_bad flag
bad_statuses = ['Charged␣Off', 'Default',
                'Does␣not␣meet␣the␣credit␣policy.␣Status:Charged␣
                   Off',
                'Late␣(31-120␣days)']
df = df.withColumn('good_bad',
    F.when(F.col('loan_status').isin(bad_statuses), 0).otherwise
       (1))

# Target 2: LGD recovery rate (charged-off loans only)
df = df.withColumn('recovery_rate',
    F.when(F.col('loan_status') == 'Charged␣Off',
           F.col('recoveries') / F.col('funded_amnt'))
    .otherwise(F.lit(None)))

# Target 3: EAD credit conversion factor
```

```
df = df.withColumn('ccf',
    F.when(F.col('loan_status') == 'Charged␣Off',
           (F.col('funded_amnt') - F.col('total_pymnt'))
           / F.col('funded_amnt'))
    .otherwise(F.lit(None)))

# OOT split flag (file NOT sorted by date)
df = df.withColumn('dataset_split',
    F.when(F.col('issue_year') <= 2015, 'train').otherwise('oot')
       )

# Drop leakage columns AFTER targets are created
df = df.drop(*LEAKAGE_COLS)
```

5.4 Performance Benchmarks

Table 5.1: PySpark job performance on single-node Docker Compose

Job	Rows Processed	Wall-clock Time
DAG-01 Ingestion (column-projected)	2,260,668	≈7 seconds
DAG-01 Ingestion (all 151 columns)	2,260,668	≈60 seconds
DAG-02 Cleaning	2,260,668	≈45 seconds
DAG-02b WoE Encoding (training set)	831,051	≈90 seconds
DAG-05 Batch Scoring	538,515	≈30 seconds

Chapter 6

MLflow Experiment Tracking and Model Registry

6.1 Why MLflow?

MLflow provides four capabilities critical for a regulated credit risk system [8]:

- **Experiment tracking**: Every training run logs parameters, metrics, and artefacts automatically, providing a reproducible history of every model version ever trained.
- **Model registry**: Models are promoted through stages (None → Staging → Production) with explicit approvals, creating an audit trail aligned with SR 11-7 model governance requirements [12].
- **Artefact storage**: Model binaries (`.pkl` files), scorecard CSVs, calibrator JSON files, and prediction arrays are stored in MinIO and linked to their training run.
- **Reproducibility**: Each run records the Git commit hash, Python version, and dependency list, ensuring that any past model version can be exactly reproduced.

6.2 Experiments Tracked

Table 6.1: MLflow experiments and logged artefacts

Experiment	Metrics Logged	Artefacts
PD-Scorecard	AUC, Gini, KS, Brier, HL stat, mean PD, intercept δ, n_features	scorecard.csv, pd_calibrator.json pd_pred_*.npy
LGD-Stage1	AUC, precision, recall, F1	lgd_stage1.pkl
LGD-Stage2	MAE, RMSE, R^2	lgd_stage2.pkl
EAD-CCF	MAE, RMSE, R^2	ead_model.pkl, ead_scaler.pkl
Portfolio-EL	Total EAD, total EL, mean LGD, EL rate	expected_loss_test.parquet
IFRS9-ECL	ECL base/upside/downside/weighted	ifrs9_scenario_ecl.parquet
Basel-Capital	Total RWA, min capital, capital ratio	capital_requirements table

6.3 Model Registry Promotion Workflow

1. DAG-03 trains the PD model and registers version n with stage = `None`.
2. An automated quality gate checks: AUC > 0.65, decile ordering monotonic, Brier ≤ 0.20. If all pass, stage is promoted to `Staging`.
3. A human reviewer (model owner or risk committee) inspects the MLflow run details, the reliability diagram, and the champion/challenger comparison.
4. If approved, the model is promoted to `Production`. The previous Production version is archived.
5. DAG-05 always loads the current `Production` version for batch scoring.

Chapter 7

Real-Time Scoring Service

7.1 FastAPI Architecture

The scoring service is a FastAPI application that loads the Production model artefacts at startup and serves real-time credit decisions via a REST endpoint. The service is containerised and starts as part of the Docker Compose stack.

Endpoint: `POST /score`
Input: JSON object with application-time loan features
Output: Full credit decision payload

Listing 7.1: FastAPI scoring endpoint response

```
{
  "loan_id": "LC123456",
  "pd": 0.0421,
  "lgd": 0.6200,
  "ead": 12500.00,
  "expected_loss": 326.55,
  "credit_score": 672,
  "risk_class": "BB",
  "annualized_roi": 0.0387,
  "decision": "APPROVE",
  "ifrs9_stage": 1,
  "adverse_action_codes": [],
  "model_version": "2.0",
  "calc_date": "2026-06-07"
}
```

7.2 Feature Pipeline at Serve Time

A key engineering challenge in production ML systems is maintaining consistency between training and serving feature transformations [9]. In this system:

1. The WoE bin boundaries computed during DAG-02b are stored in the `features.woe_bins` table.
2. The FastAPI model loader reads these bin boundaries at startup and caches them in memory.
3. For each incoming loan application, the service applies the same WoE binning logic that was used during training, ensuring the feature vector at serve time is identical in structure to the feature vector used to fit the model.
4. Redis caches the WoE-encoded feature vector for repeat applicants, reducing median latency from ≈15ms to ≈2ms for cached requests.

7.3 Performance Target

The API p99 latency target is < 200ms under peak load, consistent with the operational requirement for real-time loan origination decision systems [7]. Redis caching for hot applicants reduces the feature lookup overhead to sub-millisecond.

Chapter 8

Monitoring Infrastructure

8.1 Prometheus Metrics

DAG-06 pushes the following metrics to the Prometheus endpoint after each daily run:

Table 8.1: Prometheus metrics published by DAG-06

Metric	Type	Alert Threshold
`credit_score_psi`	Gauge	> 0.25
`feature_csi{feature="..."}`	Gauge	> 0.25
`model_gini_oot`	Gauge	< 0.32 (baseline -5pp)
`model_ks_oot`	Gauge	< 0.20
`model_brier_oot`	Gauge	> 0.25
`api_latency_p99_ms`	Gauge	> 200
`pipeline_run_success`	Counter	Any failure
`data_quality_pass_rate`	Gauge	$< 100\%$

8.2 Grafana Dashboards

Four Grafana dashboards are provisioned automatically from JSON configuration files in `infrastructure/grafana/dashboards/`:

1. **Credit Risk Portfolio** — Total EAD, EL rate, IFRS 9 stage distribution, mean PD over time.
2. **PSI/CSI Monitoring** — Score PSI trend, CSI per variable (traffic-light RAG status), champion/challenger Gini comparison.
3. **Pipeline Health** — DAG run status, data quality pass rate, row counts at each schema.
4. **API Performance** — Request rate, p50/p95/p99 latency, error rate.

8.3 Alert Routing

When any metric crosses its alert threshold, Grafana fires an alert via the configured notification channel (email, Slack, or PagerDuty). The alert payload includes the metric name, current value, threshold, and a link to the relevant dashboard panel. The alert is automatically assigned to the model owner (Credit Risk Analytics team) for investigation.

Chapter 9

Verified Pipeline Run Results

All six DAGs were executed end-to-end on the real `accepted_2007_to_2018Q4.csv` dataset with zero errors on 2026-06-07 (v4 verification run). Table 9.1 summarises the verified outputs at each pipeline stage.

Table 9.1: Verified pipeline outputs (v4 run, 2026-06-07)

DAG	**Key Output**	**Verified Value**
DAG-01 Ingestion	Rows loaded	2,260,668
	Ingest time (column-projected)	≈7 seconds
DAG-02 Cleaning	Columns after cleaning	94
	Training rows	831,051 (DR 18.62%)
	OOT rows	538,515 (DR 25.27%)
DAG-02b Feature Eng.	WoE dummy variables	56
	Raw predictors encoded	16
DAG-03 PD Training	Significant features retained	39 of 56
	AUC (OOT)	0.694
	Gini (OOT)	0.387
	KS (OOT)	0.281
	Brier Score (calibrated)	0.172
	Calibration shift δ	+0.3204
	HL statistic (pre/post calib.)	10,039 → 403 (−96%)
DAG-04 LGD/EAD	LGD MAE	≈5%
	EAD MAE	≈14%
	Mean LGD (long-run)	92.19%
	Downturn LGD (Basel)	93.76%
DAG-05 Batch Scoring	Total EAD	$3.26B
	12-month EL	$0.808B (19.93%)
	IFRS 9 Base ECL (lifetime)	$1.601B
	IFRS 9 Weighted ECL	$1.646B
	Lifetime vs 12m uplift	+$838M (+104%)
	Basel III RWA	$14.07B
	Basel III Min Capital	$1.13B
DAG-06 Monitoring	Score PSI	0.282 (**ALERT**)
	Champion Gini	0.387 (Keep champion)
	Charts generated	24 PNG files

Chapter 10

Discussion

10.1 Key Design Decisions and Their Rationale

Column-projected ingestion. Reading only 26 of 151 columns at ingest time reduces Spark load time from 60 seconds to 7 seconds. This is not just a performance optimisation — it is also good data engineering practice: columns that are never used should not be loaded, transferred, or stored.

Leakage columns used before dropping. Post-application leakage columns (`total_pymnt`, `recoveries`) must be used to construct the LGD and EAD targets before they are dropped from the feature set. The cleaning DAG enforces this ordering: target variables are created in the first transformation step, and leakage columns are dropped in the final step.

WoE fitted exclusively on the training set. The OOT split flag is set in DAG-02 and the WoE fitting step in DAG-02b explicitly filters to `dataset_split = 'train'` before computing bin statistics. This is the single most important data discipline in the pipeline: any leakage of OOT information into the WoE mapping would cause the model to be evaluated on data that influenced its feature encoding, invalidating the out-of-time validation results.

Missing values as WoE bins. Rather than imputing or dropping missing values, the pipeline assigns them to a dedicated WoE bin. This preserves the behavioural signal in missingness — a borrower with no delinquency record in the system is a different credit risk profile from one with an explicitly zero delinquency record [6].

Statsmodels over scikit-learn for PD training. Scikit-learn's `LogisticRegression` does not natively provide Wald-test p-values with appropriate confidence intervals. Statsmodels' `Logit` does, and these p-values are required by EBA/GL/2017/16 for regulatory documentation of IRB models [2]. The feature selection loop that iteratively removes variables with the highest max p-value is only possible with statsmodels' output.

Artefact-based DAG dependencies. Each DAG depends on the successful creation of artefacts by the previous DAG, not on the previous DAG's state. This means DAGs

can be re-run independently (e.g., re-running only the LGD training without re-running the WoE encoding), which is essential for iterative model development.

10.2 Limitations and Next Steps

1. **Score PSI = 0.282 (ALERT):** The monitoring DAG correctly detects that the 2016–2018 applicant population has shifted materially from the 2007–2015 training window. The next pipeline update should trigger a DAG-03 re-run with 2016–2018 training data and compare the resulting model metrics via the champion/challenger framework.
2. **Airflow DAG productionisation:** The six DAGs are currently scaffolded with the correct logic but not yet fully connected to the live PostgreSQL tables and MinIO buckets. The final productionisation step involves replacing the file-path-based artefact handoffs with proper database reads/writes.
3. **Great Expectations integration:** Data quality validation with Great Expectations is designed into the architecture but not yet fully implemented. Each DAG should have an attached validation suite that fails the task if data quality assertions are violated.
4. **Grafana dashboards:** The dashboard JSON configurations exist in the repository but require connection to a live Prometheus data source. The final deployment step is to verify that all four dashboards render correctly with live pipeline data.

Chapter 11

Conclusion

This paper has described the design and implementation of a six-DAG Apache Airflow data engineering pipeline that transforms 2.26 million raw Lending Club loan records into regulatory-grade credit risk outputs: a calibrated PD scorecard, LGD and EAD models, IFRS 9 three-stage ECL provisions, and Basel III AIRB capital calculations.

The six pipelines — ingestion, cleaning, feature engineering, model training (PD and LGD/EAD separately), batch scoring, and daily monitoring — each address a distinct engineering concern and produce well-defined artefacts that feed the next stage. Together they demonstrate that the gap between a trained credit risk model and a production data engineering system is large and consequential: column-projected ingest, OOT-split-only WoE fitting, post-ingest leakage removal, statsmodels for regulatory p-values, artefact-based DAG dependencies, MLflow governance gates, and daily PSI alerting are all engineering decisions that directly affect the correctness and regulatory acceptability of the downstream model outputs.

The v4 pipeline run verified all six DAGs end-to-end on the real dataset with zero errors, producing 24 visualisations, all model artefacts, and the complete regulatory output stack. The Score PSI of 0.282 (ALERT) in the monitoring DAG correctly identifies the need for model rebuild on the 2016–2018 population — demonstrating that the monitoring infrastructure is functioning as designed.

Bibliography

[1] Astronomer. (2021). *The future of banking: How Apache Airflow helps*. https://www.astronomer.io/blog/future-of-banking-apache-airflow/

[2] European Banking Authority (EBA). (2017). *Guidelines on PD estimation, LGD estimation and the treatment of defaulted exposures* (EBA/GL/2017/16). https://www.eba.europa.eu

[3] GARP. (2021). Probability of default: Pros and cons of the population stability index. *GARP Risk Intelligence*. https://www.garp.org/risk-intelligence/credit/probability-of-default-pros-and-cons-of-the-population-stability-index

[4] Great Expectations. (2024). *GX Core: Open source data quality platform*. https://greatexpectations.io

[5] Kaggle. (2018). *All Lending Club loan data* [Dataset]. https://www.kaggle.com/datasets/wordsforthewise/lending-club

[6] Lund University. (2021). *Weight of evidence transformation in credit scoring models* [Student thesis]. https://lup.lub.lu.se/student-papers/record/9066332/file/9067075.pdf

[7] MathWorks. (n.d.). *Credit risk modeling: Importance and key components*. https://www.mathworks.com/discovery/credit-risk-modeling.html

[8] MLflow. (2026). *MLflow model registry documentation*. https://mlflow.org/docs/latest/ml/model-registry/

[9] Sculley, D., et al. (2015). Hidden technical debt in machine learning systems. *Proceedings of NeurIPS*. https://papers.neurips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems.pdf

[10] Siddiqi, N. (2006). *Credit risk scorecards: Developing and implementing intelligent credit scoring*. John Wiley & Sons.

[11] Apache Spark. (2026). *Apache Spark – Unified engine for large-scale data analytics*. https://spark.apache.org

[12] Board of Governors of the Federal Reserve System. (2011). *Supervisory guidance on model risk management (SR 11-7)*. https://www.federalreserve.gov/frrs/guidance/supervisory-guidance-on-model-risk-management.htm